



Multi-Warehouse Stock Transfer Management System

Design and Implementation of a Database Management System
DBMS Course Project 09 · Project Report

Name	Katamreddy Shreenandh
Roll number	25WU0102119
Section	B.Tech CSE (AI & ML), Panthers
Course	Database Management Systems
Faculty	Dr. Kiran Mayee Adavala
Academic year	2026–27
Institution	Woxsen University, Hyderabad

Contents

1. Cover page
2. Abstract
3. Introduction and problem statement
4. Objectives and scope
5. Software and hardware requirements
6. ER diagram
7. Relational schema and normalization
8. Data dictionary
9. SQL commands used (DDL, DML) with sample outputs
10. Queries with outputs
11. UI design and screenshots
12. Implementation details
13. Testing
14. Conclusion and future enhancements
15. References
16. Appendix: GitHub repository link

2. Abstract

Companies that run several warehouses lose track of stock between the moment goods are received and the moment they are dispatched, and the worst gap is the inter-warehouse transfer that has left one site but not yet been confirmed at the other. This project designs and implements a relational database, `stock_transfer_db`, that records every physical stock movement (receipt, put-away, transfer, dispatch and damage) across warehouses, zones and bins.

The conceptual model was drawn as a Chen ER diagram and mapped to 12 relations in Third Normal Form (also BCNF), implemented in MySQL 8.0 with 19 foreign keys, 8 CHECK constraints, 3 triggers and one view. Stock on hand is never stored; the view `v_bin_stock` derives it from the movement tables, so a transfer is subtracted from the source bin once it is in transit and credited to the destination only when it is confirmed. A Python desktop application (PySide6 + mysql-connector-python) connects live to the database and supports viewing, inserting and deleting records, guided stock workflows and six reports, with every write executed as a single transaction.

3. Introduction and Problem Statement

A company operating warehouses in Hyderabad, Bengaluru and Pune must be able to trace any unit of stock from the supplier delivery, into the bin where it is put away, across transfers between warehouses, to dispatch or write-off. When this is kept in spreadsheets the chain breaks: stock in transit is counted at both ends or at neither, damaged units are discovered late, and reorder levels are noticed only after a shortage.

Problem statement. Design and implement a database management system that records every stock movement between suppliers, warehouses, bins and customers so that stock balances per bin, pending transfers, bin utilisation and reorder needs can be reported accurately, and so that the business rules (for example, that a transfer is credited to the destination only after confirmation) are enforced by the database itself.

Users

Role	Responsibility
Warehouse Manager	Approves transfers; reviews stock and reorder reports
Store Keeper	Records receipts; puts stock away into bins
Transfer Officer	Initiates transfers; confirms arrival at the destination
Auditor	Read-only access to balances, ageing and damage records

Functional requirements

ID	Requirement
FR-01	Record a receipt of goods from a supplier
FR-02	Put received stock away into a bin
FR-03	Search stock by product, bin or warehouse
FR-04	Initiate an inter-warehouse transfer
FR-05	Confirm arrival before the destination is credited
FR-06	Dispatch stock to an external destination
FR-07	Record damaged stock
FR-08	Report on stock levels, pending transfers and reorder needs

Business rules

1. Source and destination warehouse of a transfer must differ.
2. The destination is credited only once a transfer is CONFIRMED.
3. Bin codes are unique.
4. All movement quantities must be positive.
5. Transfer and dispatch quantities may not exceed the available stock in the source bin.

4. Objectives and Scope

Objectives

1. Model every warehouse, zone, bin, product and stock movement as one ER schema.
2. Translate the model into a normalized (3NF) relational schema with keys.
3. Enforce transfer, bin and quantity rules as database constraints and triggers.
4. Derive stock balances from movements so they can never drift out of sync.
5. Report warehouse stock, pending transfers, bin use and reorder needs.
6. Provide a working user interface for viewing, inserting and deleting records.

Scope

In scope: warehouses, zones, bins, products, suppliers, receiving, put-away, inter-warehouse transfer with confirmation, dispatch, damage, stock and reorder reporting.

Out of scope: purchasing, invoicing, carrier tracking, pricing and payroll.

5. Software and Hardware Requirements

Software

Component	Version / detail
Operating system	Windows 10/11 (64-bit)
Database server	MySQL Community Server 8.0 (8.0.46 used), InnoDB engine
Database client	MySQL command-line client / MySQL Workbench
Programming language	Python 3.12 or later
UI toolkit	PySide6-Essentials 6.11.2 (Qt 6)
Database driver	mysql-connector-python 26.7.0
Credential storage	keyring 25.7.0 (Windows Credential Manager)
Version control	Git and GitHub
IDE	PyCharm

Hardware (minimum)

Component	Requirement
Processor	Dual-core 64-bit, 2 GHz or faster
Memory	4 GB RAM (8 GB recommended)
Storage	2 GB free for MySQL, Python and the project
Display	1366 × 768 or higher

6. ER Diagram

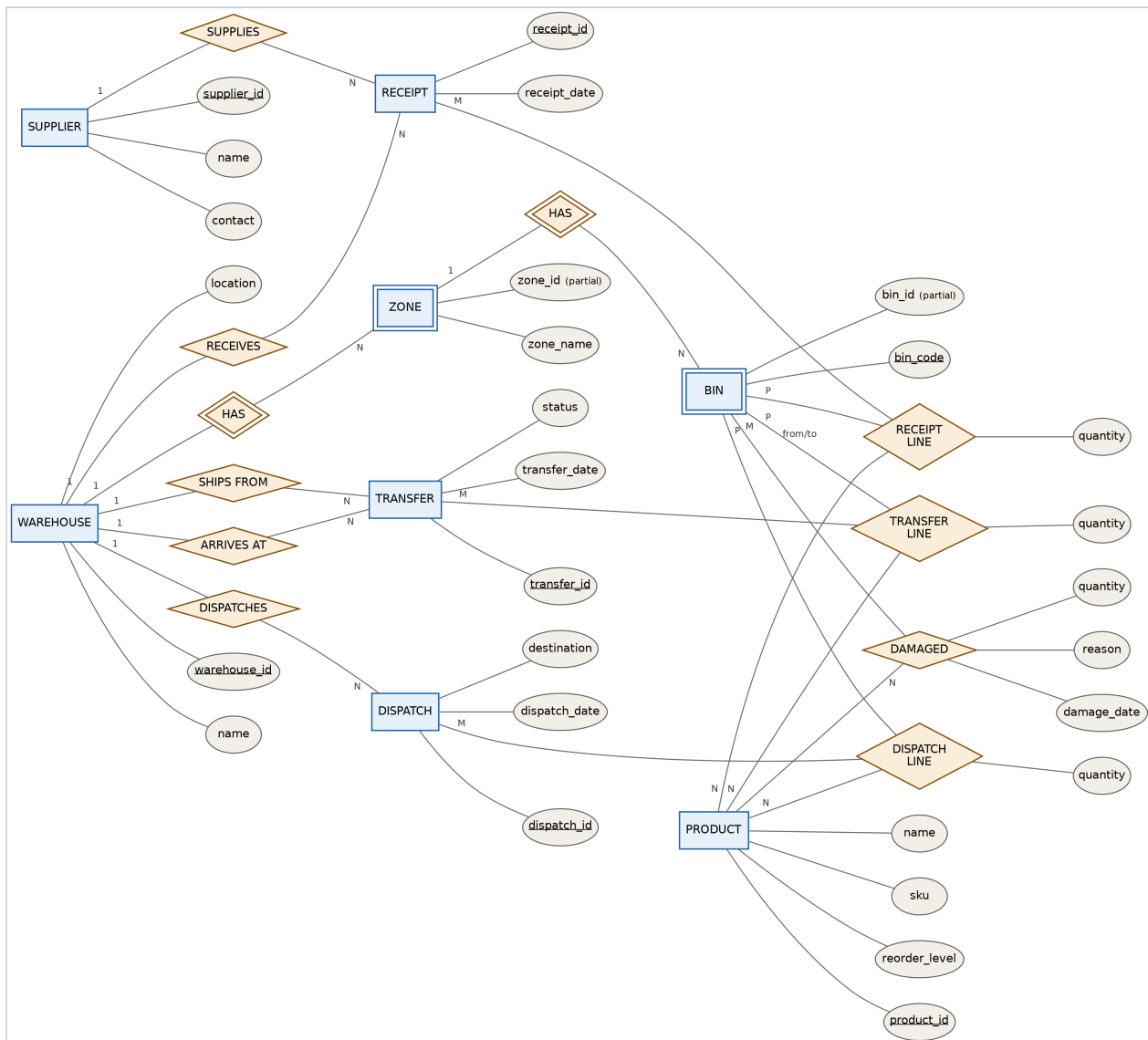


Figure 6.1 – ER diagram in Chen notation

Six strong entities (Warehouse, Supplier, Product, Receipt, Transfer, Dispatch), two weak entities (Zone, identified by its warehouse; Bin, identified by its zone) and four relationships that carry a quantity (Receipt Line, Transfer Line, Dispatch Line, Damaged). Transfer relates to Warehouse twice (ships from / arrives at), and Transfer Line relates to Bin twice (from bin / to bin).

7. Relational Schema and Normalization

7.1 Relational schema

```

Supplier      (supplier_id, name, contact)
Warehouse     (warehouse_id, name, location)
Product       (product_id, name, sku, reorder_level)
Zone          (zone_id, zone_name, warehouse_id -> Warehouse)
Bin           (bin_id, bin_code, zone_id -> Zone)
Receipt       (receipt_id, receipt_date, supplier_id -> Supplier, warehouse_id -> Warehouse)
Transfer      (transfer_id, status, transfer_date,
              source_warehouse_id -> Warehouse, dest_warehouse_id -> Warehouse)
Dispatch      (dispatch_id, destination, dispatch_date, warehouse_id -> Warehouse)
Receipt_Line  (receipt_id -> Receipt, bin_id -> Bin, product_id -> Product, quantity)
Transfer_Line (transfer_id -> Transfer, source_bin_id -> Bin, product_id -> Product,
              dest_bin_id -> Bin [nullable], quantity)
Dispatch_Line (dispatch_id -> Dispatch, bin_id -> Bin, product_id -> Product, quantity)
Damaged       (bin_id -> Bin, product_id -> Product, damage_date, quantity, reason)

```

7.2 Mapping decisions

- 1:N relationships become a foreign key on the N side (e.g. Zone.warehouse_id, Receipt.supplier_id).
- Weak entities Zone and Bin use a NOT NULL foreign key with ON DELETE CASCADE; a surrogate key keeps every foreign key single-column.
- Relationships with a quantity attribute (lines, damage) become their own relations with composite primary keys.
- Stock balance is not stored; it is computed by the view v_bin_stock.

7.3 Functional dependencies and normal forms

Relation	Key	Functional dependencies
Supplier	supplier_id	supplier_id → name, contact; name → supplier_id (candidate key)
Warehouse	warehouse_id	warehouse_id → name, location
Product	product_id	product_id → name, sku, reorder_level; sku → product_id
Zone	zone_id	zone_id → zone_name, warehouse_id; (warehouse_id, zone_name) → zone_id
Bin	bin_id	bin_id → bin_code, zone_id; bin_code → bin_id
Receipt	receipt_id	receipt_id → receipt_date, supplier_id, warehouse_id
Transfer	transfer_id	transfer_id → status, transfer_date, source_warehouse_id, dest_warehouse_id
Dispatch	dispatch_id	dispatch_id → destination, dispatch_date, warehouse_id
Receipt_Line	(receipt_id, bin_id, product_id)	key → quantity
Transfer_Line	(transfer_id, source_bin_id, product_id)	key → dest_bin_id, quantity
Dispatch_Line	(dispatch_id, bin_id, product_id)	key → quantity
Damaged		key → quantity, reason

	(bin_id, product_id, damage_date)	
--	--------------------------------------	--

1NF: every attribute is atomic and every relation has a primary key. **2NF:** in the relations with composite keys the non-key attributes (quantity, dest_bin_id, reason) depend on the whole key, never on part of it. **3NF:** no non-key attribute depends on another non-key attribute; for example a bin's warehouse is reached through Zone instead of being stored on Bin, and supplier or warehouse names are not repeated on Receipt. **BCNF:** every determinant listed above is a candidate key, so all 12 relations are also in BCNF.

8. Data Dictionary

8.1 Supplier

Column	Data type	Constraints
supplier_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL, UNIQUE
contact	VARCHAR(100)	NOT NULL

8.2 Warehouse

Column	Data type	Constraints
warehouse_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL, UNIQUE
location	VARCHAR(150)	NOT NULL

8.3 Product

Column	Data type	Constraints
product_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
sku	VARCHAR(30)	NOT NULL, UNIQUE
reorder_level	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0

8.4 Zone

Column	Data type	Constraints
zone_id	INT	PK, AUTO_INCREMENT
zone_name	VARCHAR(50)	NOT NULL; UNIQUE with warehouse_id
warehouse_id	INT	NOT NULL, FK $\rightarrow$ Warehouse (CASCADE)

8.5 Bin

Column	Data type	Constraints
bin_id	INT	PK, AUTO_INCREMENT
bin_code	VARCHAR(20)	NOT NULL, UNIQUE
zone_id	INT	NOT NULL, FK $\rightarrow$ Zone (CASCADE)

8.6 Receipt

Column	Data type	Constraints
receipt_id	INT	PK, AUTO_INCREMENT
receipt_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
supplier_id	INT	NOT NULL, FK $\rightarrow$ Supplier (RESTRICT)
warehouse_id	INT	NOT NULL, FK $\rightarrow$ Warehouse (RESTRICT)

8.7 Transfer

Column	Data type	Constraints
transfer_id	INT	PK, AUTO_INCREMENT
status	VARCHAR(12)	NOT NULL, DEFAULT 'PENDING', CHECK IN (PENDING, IN_TRANSIT, CONFIRMED, CANCELLED)
transfer_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
source_warehouse_id	INT	NOT NULL, FK → Warehouse
dest_warehouse_id	INT	NOT NULL, FK → Warehouse; CHECK ≠ source

8.8 Dispatch

Column	Data type	Constraints
dispatch_id	INT	PK, AUTO_INCREMENT
destination	VARCHAR(150)	NOT NULL
dispatch_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
warehouse_id	INT	NOT NULL, FK → Warehouse (RESTRICT)

8.9 Receipt_Line

Column	Data type	Constraints
receipt_id	INT	PK part, FK → Receipt (CASCADE)
bin_id	INT	PK part, FK → Bin
product_id	INT	PK part, FK → Product
quantity	INT	NOT NULL, CHECK > 0

8.10 Transfer_Line

Column	Data type	Constraints
transfer_id	INT	PK part, FK → Transfer (CASCADE)
source_bin_id	INT	PK part, FK → Bin
product_id	INT	PK part, FK → Product
dest_bin_id	INT	NULL until put-away, FK → Bin, CHECK ≠ source_bin_id
quantity	INT	NOT NULL, CHECK > 0

8.11 Dispatch_Line

Column	Data type	Constraints
dispatch_id	INT	PK part, FK → Dispatch (CASCADE)
bin_id	INT	PK part, FK → Bin
product_id	INT	PK part, FK → Product
quantity	INT	NOT NULL, CHECK > 0

8.12 Damaged

Column	Data type	Constraints
bin_id	INT	PK part, FK → Bin
product_id	INT	PK part, FK → Product
damage_date	DATE	PK part, DEFAULT CURRENT_DATE
quantity	INT	NOT NULL, CHECK > 0
reason	VARCHAR(200)	NOT NULL

8.13 View v_bin_stock

Columns: warehouse_id, warehouse_name, zone_name, bin_id, bin_code, product_id, sku, product_name, received, transferred_in, transferred_out, dispatched, damaged, on_hand.
 $\text{on_hand} = \text{received} + \text{transferred_in} - \text{transferred_out} - \text{dispatched} - \text{damaged}$.

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 DDL – database and tables

```
CREATE DATABASE IF NOT EXISTS stock_transfer_db
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_0900_ai_ci;

USE stock_transfer_db;
```

```
CREATE TABLE Zone (
  zone_id      INT          NOT NULL AUTO_INCREMENT,
  zone_name    VARCHAR(50)  NOT NULL,
  warehouse_id INT          NOT NULL,
  CONSTRAINT pk_zone PRIMARY KEY (zone_id),
  CONSTRAINT uq_zone_per_warehouse UNIQUE (warehouse_id, zone_name),
  CONSTRAINT fk_zone_warehouse FOREIGN KEY (warehouse_id)
    REFERENCES Warehouse (warehouse_id)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB;
```

```
CREATE TABLE Transfer (
  transfer_id      INT          NOT NULL AUTO_INCREMENT,
  status           VARCHAR(12) NOT NULL DEFAULT 'PENDING',
  transfer_date    DATE         NOT NULL DEFAULT (CURRENT_DATE),
  source_warehouse_id INT      NOT NULL,
  dest_warehouse_id INT      NOT NULL,
  CONSTRAINT pk_transfer PRIMARY KEY (transfer_id),
  CONSTRAINT chk_transfer_status
    CHECK (status IN ('PENDING', 'IN_TRANSIT', 'CONFIRMED', 'CANCELLED')),
  CONSTRAINT chk_transfer_diff_warehouse
    CHECK (source_warehouse_id <> dest_warehouse_id),
  -- RESTRICT (not CASCADE) is required: MySQL rejects CHECKs on columns used by cascading FK
  actions.
  CONSTRAINT fk_transfer_source FOREIGN KEY (source_warehouse_id)
    REFERENCES Warehouse (warehouse_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_transfer_dest FOREIGN KEY (dest_warehouse_id)
    REFERENCES Warehouse (warehouse_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE = InnoDB;
```

```
CREATE TABLE Transfer_Line (
  transfer_id  INT NOT NULL,
  source_bin_id INT NOT NULL,
  product_id  INT NOT NULL,
  dest_bin_id  INT NULL,      -- NULL until put-away at the destination warehouse
  quantity    INT NOT NULL,
  CONSTRAINT pk_transfer_line PRIMARY KEY (transfer_id, source_bin_id, product_id),
  CONSTRAINT chk_transfer_line_qty CHECK (quantity > 0),
  CONSTRAINT chk_transfer_line_bins
    CHECK (dest_bin_id IS NULL OR dest_bin_id <> source_bin_id),
  CONSTRAINT fk_tl_transfer FOREIGN KEY (transfer_id)
    REFERENCES Transfer (transfer_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  -- RESTRICT on both bin FKs: their columns appear in chk_transfer_line_bins.
  CONSTRAINT fk_tl_source_bin FOREIGN KEY (source_bin_id)
    REFERENCES Bin (bin_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_tl_dest_bin FOREIGN KEY (dest_bin_id)
    REFERENCES Bin (bin_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
```

```

        CONSTRAINT fk_tl_product FOREIGN KEY (product_id)
        REFERENCES Product (product_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
    ) ENGINE = InnoDB;

```

The full script with all 12 tables is `stock_transfer_db.sql` in the repository.

9.2 DDL – trigger

```

CREATE TRIGGER trg_transfer_confirm_bu
BEFORE UPDATE ON Transfer
FOR EACH ROW
BEGIN
    IF NEW.status = 'CONFIRMED' AND EXISTS (
        SELECT 1 FROM Transfer_Line
        WHERE transfer_id = NEW.transfer_id AND dest_bin_id IS NULL
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot confirm transfer: every line needs a dest_bin_id (put-away)
first';
    END IF;
END$$

```

9.3 DDL – view

```

CREATE VIEW v_bin_stock AS
SELECT w.warehouse_id,
       w.name          AS warehouse_name,
       z.zone_name,
       b.bin_id,
       b.bin_code,
       p.product_id,
       p.sku,
       p.name          AS product_name,
       SUM(CASE WHEN m.movement = 'RECEIPT'      THEN m.qty ELSE 0 END) AS received,
       SUM(CASE WHEN m.movement = 'TRANSFER_IN'  THEN m.qty ELSE 0 END) AS transferred_in,
       SUM(CASE WHEN m.movement = 'TRANSFER_OUT' THEN -m.qty ELSE 0 END) AS transferred_out,
       SUM(CASE WHEN m.movement = 'DISPATCH'    THEN -m.qty ELSE 0 END) AS dispatched,
       SUM(CASE WHEN m.movement = 'DAMAGED'      THEN -m.qty ELSE 0 END) AS damaged,
       SUM(m.qty) AS on_hand
FROM (
    SELECT 'RECEIPT' AS movement, bin_id, product_id, quantity AS qty
    FROM Receipt_Line
    UNION ALL
    SELECT 'TRANSFER_OUT', tl.source_bin_id, tl.product_id, -tl.quantity
    FROM Transfer_Line tl
    JOIN Transfer t ON t.transfer_id = tl.transfer_id
    WHERE t.status IN ('IN_TRANSIT', 'CONFIRMED')
    UNION ALL
    SELECT 'TRANSFER_IN', tl.dest_bin_id, tl.product_id, tl.quantity
    FROM Transfer_Line tl
    JOIN Transfer t ON t.transfer_id = tl.transfer_id
    WHERE t.status = 'CONFIRMED'
    UNION ALL
    SELECT 'DISPATCH', bin_id, product_id, -quantity
    FROM Dispatch_Line
    UNION ALL
    SELECT 'DAMAGED', bin_id, product_id, -quantity
    FROM Damaged
) AS m
JOIN Bin b      ON b.bin_id = m.bin_id
JOIN Zone z    ON z.zone_id = b.zone_id
JOIN Warehouse w ON w.warehouse_id = z.warehouse_id
JOIN Product p  ON p.product_id = m.product_id

```

```
GROUP BY w.warehouse_id, w.name, z.zone_name, b.bin_id, b.bin_code,
        p.product_id, p.sku, p.name;
```

9.4 DML – sample data

```
INSERT INTO Warehouse (warehouse_id, name, location) VALUES
(1, 'Hyderabad Central DC', 'Shamshabad, Hyderabad, Telangana'),
(2, 'Bengaluru East Hub', 'Hoskote, Bengaluru, Karnataka'),
(3, 'Pune West Depot', 'Chakan, Pune, Maharashtra');
```

```
INSERT INTO Zone (zone_id, zone_name, warehouse_id) VALUES
(1, 'Receiving Dock', 1),
(2, 'Bulk Storage', 1),
(3, 'Fast-Pick', 1),
(4, 'Bulk Storage', 2),
(5, 'Fast-Pick', 2),
(6, 'Bulk Storage', 3),
(7, 'Returns & Quarantine', 3);
```

```
INSERT INTO Transfer (transfer_id, status, transfer_date, source_warehouse_id, dest_warehouse_id)
VALUES
(1, 'CONFIRMED', '2026-08-10', 1, 2),
(2, 'IN_TRANSIT', '2026-08-14', 1, 3),
(3, 'PENDING', '2026-08-18', 2, 3);
```

```
INSERT INTO Transfer_Line (transfer_id, source_bin_id, product_id, dest_bin_id, quantity) VALUES
(1, 3, 1, 9, 300), -- LED bulbs HYD-BS-01 -> BLR-BS-02 (received)
(2, 4, 7, NULL, 500), -- Switches HYD-BS-02 -> Pune (on the road)
(3, 8, 3, NULL, 150); -- PVC pipes BLR-BS-01 -> Pune (not yet shipped)
```

9.5 Sample outputs

Row counts after loading the script:

```
+-----+-----+
| t      | n  |
+-----+-----+
| Supplier | 7 |
| Warehouse | 3 |
| Zone     | 7 |
| Bin      | 17 |
| Product  | 13 |
| Receipt  | 9 |
| Receipt_Line | 16 |
| Transfer | 8 |
| Transfer_Line | 8 |
| Dispatch | 6 |
| Dispatch_Line | 11 |
| Damaged  | 5 |
+-----+-----+
```

```
SELECT * FROM Warehouse;
```

```
+-----+-----+-----+
| warehouse_id | name | location |
+-----+-----+-----+
| 1 | Hyderabad Central DC | Shamshabad, Hyderabad, Telangana |
| 2 | Bengaluru East Hub | Hoskote, Bengaluru, Karnataka |
| 3 | Pune West Depot | Chakan, Pune, Maharashtra |
+-----+-----+-----+
```

Foreign keys present in the catalog and the triggers created:

```
+-----+
| fk_columns |
+-----+
|      19 |
+-----+
```

```
+-----+-----+-----+-----+
| TRIGGER_NAME | EVENT_MANIPULATION | EVENT_OBJECT_TABLE | ACTION_TIMING |
+-----+-----+-----+-----+
| trg_transfer_confirm_bu | UPDATE | transfer | BEFORE |
| trg_transfer_line_dest_bi | INSERT | transfer_line | BEFORE |
| trg_transfer_line_dest_bu | UPDATE | transfer_line | BEFORE |
+-----+-----+-----+-----+
```

10. Queries with Outputs

10.1 Query given during Presentation-II

Display the details of every stock transfer: transfer ID, status, source and destination warehouse names, product and quantity.

```
SELECT t.transfer_id, t.status, sw.name AS from_warehouse, dw.name AS to_warehouse,
       p.name AS product, tl.quantity
FROM Transfer t
JOIN Warehouse sw ON t.source_warehouse_id = sw.warehouse_id
JOIN Warehouse dw ON t.dest_warehouse_id = dw.warehouse_id
JOIN Transfer_Line tl ON t.transfer_id = tl.transfer_id
JOIN Product p ON tl.product_id = p.product_id;
```

transfer_id	status	from_warehouse	to_warehouse	product	quantity
1	CONFIRMED	Hyderabad Central DC	Bengaluru East Hub	LED Bulb 9W (Pack of 4)	300
2	IN_TRANSIT	Hyderabad Central DC	Pune West Depot	Modular Switch	500
3	PENDING	Bengaluru East Hub	Pune West Depot	PVC Pipe 1in x 3m	150
4	CONFIRMED	Hyderabad Central DC	Bengaluru East Hub	Copper Wire 1.5 sq mm (90m)	40
5	CONFIRMED	Bengaluru East Hub	Pune West Depot	Ball Valve 1in Brass	30
6	CANCELLED	Bengaluru East Hub	Hyderabad Central DC	CPVC Elbow 1in (Pack of 10)	50
7	IN_TRANSIT	Hyderabad Central DC	Pune West Depot	MCB 32A Single Pole	150
8	PENDING	Pune West Depot	Hyderabad Central DC	Distribution Board 8-Way	15

```

+-----+
| Tables_in_stock_transfer_db |
+-----+
| bin
| damaged
| dispatch
| dispatch_line
| product
| receipt
| receipt_line
| supplier
| transfer
| transfer_line
| v_bin_stock
| warehouse
| zone
+-----+
13 rows in set (0.05 sec)

mysql> SELECT t.transfer_id, t.status, sw.name AS from_warehouse, dw.name AS to_warehouse, p.name AS product, tl.quantity
-> FROM Transfer t
-> JOIN Warehouse sw ON t.source_warehouse_id = sw.warehouse_id
-> JOIN Warehouse dw ON t.dest_warehouse_id = dw.warehouse_id
-> JOIN Transfer_Line tl ON t.transfer_id = tl.transfer_id
-> JOIN Product p ON tl.product_id = p.product_id;
+-----+
| transfer_id | status | from_warehouse | to_warehouse | product | quantity |
+-----+
| 1 | CONFIRMED | Hyderabad Central DC | Bengaluru East Hub | LED Bulb 9W (Pack of 4) | 300 |
| 2 | IN_TRANSIT | Hyderabad Central DC | Pune West Depot | Modular Switch 6A | 500 |
| 3 | PENDING | Bengaluru East Hub | Pune West Depot | PVC Pipe 1in x 3m | 150 |
+-----+
3 rows in set (0.08 sec)

```

Figure 10.1 – Output of the given query as submitted in Presentation-II

10.2 Bins of a warehouse with their zone (join through Zone)

```

SELECT b.bin_code, z.zone_name, w.name AS warehouse FROM Bin b JOIN Zone z ON z.zone_id=b.zone_id
JOIN Warehouse w ON w.warehouse_id=z.warehouse_id WHERE w.warehouse_id=1 ORDER BY b.bin_code;

```

```

+-----+
| bin_code | zone_name | warehouse |
+-----+
| HYD-BS-01 | Bulk Storage | Hyderabad Central DC |
| HYD-BS-02 | Bulk Storage | Hyderabad Central DC |
| HYD-BS-03 | Bulk Storage | Hyderabad Central DC |
| HYD-FP-01 | Fast-Pick | Hyderabad Central DC |
| HYD-FP-02 | Fast-Pick | Hyderabad Central DC |
| HYD-RD-01 | Receiving Dock | Hyderabad Central DC |
| HYD-RD-02 | Receiving Dock | Hyderabad Central DC |
+-----+

```

10.3 Stock on hand per warehouse (view + GROUP BY)

```

SELECT warehouse_name, COUNT(DISTINCT bin_id) AS bins_used, SUM(on_hand) AS units_on_hand FROM
v_bin_stock GROUP BY warehouse_id, warehouse_name ORDER BY warehouse_id;

```

```

+-----+
| warehouse_name | bins_used | units_on_hand |
+-----+
| Hyderabad Central DC | 5 | 2614 |
| Bengaluru East Hub | 5 | 1312 |
| Pune West Depot | 3 | 1200 |
+-----+

```

10.4 Products below reorder level (LEFT JOIN + HAVING)

```

SELECT p.sku, p.name, p.reorder_level, COALESCE(SUM(v.on_hand),0) AS on_hand FROM Product p LEFT
JOIN v_bin_stock v ON v.product_id=p.product_id GROUP BY p.product_id, p.sku, p.name,
p.reorder_level HAVING on_hand < p.reorder_level ORDER BY p.sku;

```

```

+-----+
| sku | name | reorder_level | on_hand |
+-----+

```

ELE-FAN-1200	Ceiling Fan 1200mm	40	20
PLB-VAL-025	Ball Valve 1in Brass	80	65

10.5 Transfers by status

```
SELECT status, COUNT(*) AS transfers FROM Transfer GROUP BY status ORDER BY status;
```

status	transfers
CANCELLED	1
CONFIRMED	3
IN_TRANSIT	2
PENDING	2

10.6 Damage log with warehouse and product

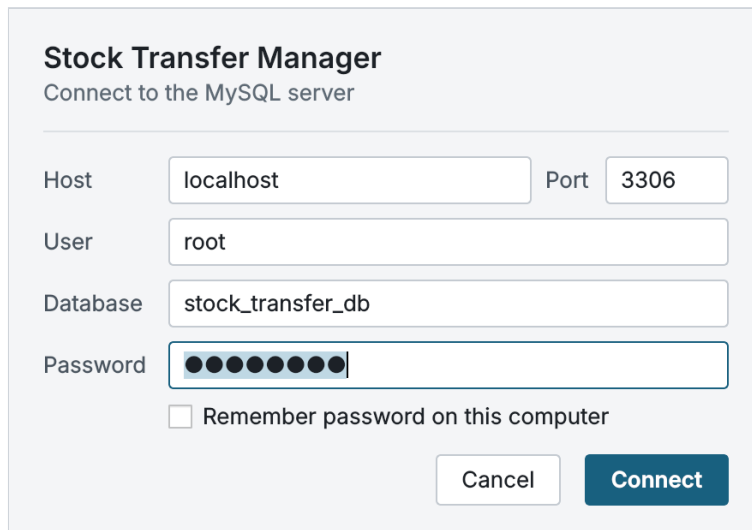
```
SELECT w.name AS warehouse, p.sku, d.quantity, d.reason FROM Damaged d JOIN Bin b ON
b.bin_id=d.bin_id JOIN Zone z ON z.zone_id=b.zone_id JOIN Warehouse w ON
w.warehouse_id=z.warehouse_id JOIN Product p ON p.product_id=d.product_id ORDER BY d.damage_date;
```

warehouse	sku	quantity	reason
Hyderabad Central DC	ELE-LED-009	24	Cartons crushed during unloading
Bengaluru East Hub	PLB-PVC-025	12	Pipes cracked by forklift impact
Hyderabad Central DC	TLS-DRL-018	2	Battery packs swollen on arrival
Pune West Depot	ELE-FAN-1200	3	Fan blades bent in transit
Bengaluru East Hub	PNT-BLK-001	6	Tins dented, lids leaking

11. UI Design and Screenshots

The Stock Transfer Manager is a Windows desktop application. A sidebar groups the pages from everyday work to setup: **Tasks** (receive, new transfer, process transfers, dispatch, record damage), **Look up** (bin stock, bin search, reports), **Movement records**, **Locations** and **Products & suppliers**. Every grid shows foreign keys as names, supports sorting and filtering, and the **SQL activity** panel at the bottom shows each statement sent with START TRANSACTION / COMMIT / ROLLBACK.

11.1 Connection



Stock Transfer Manager
Connect to the MySQL server

Host: localhost Port: 3306

User: root

Database: stock_transfer_db

Password: [masked]

☐ Remember password on this computer

Cancel Connect

Figure 11.1 – Login dialog (connection details prefilled; password never stored in the project)

11.2 View records

File

Edit

View

Help

TASKS

Receive stock

New transfer

Process transfers4

Dispatch stock

Record damage

LOOK UP

Bin stock19

Bin search

Reports

MOVEMENT RECORDS

Receipts9

Receipt lines16

Transfers8

Transfer lines8

Dispatches6

Dispatch lines11

Damage records5

LOCATIONS

Warehouses3

Zones7

Bins17

PRODUCTS & SUPPLIERS

Bin stock

Current stock in every bin, calculated from all movements (read-only).

View v_bin_stock · key (bin_id, product_id)

All columns

Filter rows (Ctrl+F)

Refresh

Export CSV

On hand total 5,126

Warehouse	Bin	SKU	Product	On hand	Received	Transfer in	Transfer out
Hyderabad Central DC	HYD-BS-01	ELE-LED-009	LED Bulb 9W (Pack of 4)	726	1,200	0	0
Hyderabad Central DC	HYD-BS-02	ELE-SWT-006	Modular Switch 6A	1,600	2,500	0	0
Hyderabad Central DC	HYD-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	50	120	0	0
Hyderabad Central DC	HYD-FP-01	ELE-MCB-032	MCB 32A Single Pole	130	400	0	0
Hyderabad Central DC	HYD-FP-02	TLS-BIT-005	Hammer Drill Bit Set 5pc	70	70	0	0
Hyderabad Central DC	HYD-FP-02	TLS-DRL-018	Cordless Drill 18V	38	40	0	0
Bengaluru East Hub	BLR-BS-01	PLB-PVC-025	PVC Pipe 1in x 3m	588	600	0	0
Bengaluru East Hub	BLR-BS-02	ELE-LED-009	LED Bulb 9W (Pack of 4)	300	0	300	0
Bengaluru East Hub	BLR-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	40	0	40	0
Bengaluru East Hub	BLR-BS-03	PLB-ELB-025	CPVC Elbow 1in (Pack of 10)	180	300	0	0
Bengaluru East Hub	BLR-FP-01	PLB-VAL-025	Ball Valve 1in Brass	35	90	0	0

SQL activity

Clear

Statements sent by inserts, deletes and workflows appear here.

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46

19 rows

Figure 11.2 – Bin stock view (v_bin_stock): 19 rows, on-hand total 5,126

File

Edit

View

Help

TASKS

Receive stock

New transfer

Process transfers4

Dispatch stock

Record damage

LOOK UP

Bin stock19

Bin search

Reports

MOVEMENT RECORDS

Receipts9

Receipt lines16

Transfers8

Transfer lines8

Dispatches6

Dispatch lines11

Damage records5

LOCATIONS

Warehouses3

Zones7

Bins17

PRODUCTS & SUPPLIERS

Transfers

Transfer headers: from which warehouse to which, and their status.

Table transfer · key (transfer_id)

All columns

Filter rows (Ctrl+F)

Refresh

Export CSV

Delete

New

Transfer	Status	Date	From warehouse	To warehouse
1	CONFIRMED	2026-08-10	Hyderabad Central DC	Bengaluru East Hub
2	IN_TRANSIT	2026-08-14	Hyderabad Central DC	Pune West Depot
3	PENDING	2026-08-18	Bengaluru East Hub	Pune West Depot
4	CONFIRMED	2026-08-28	Hyderabad Central DC	Bengaluru East Hub
5	CONFIRMED	2026-09-08	Bengaluru East Hub	Pune West Depot
6	CANCELLED	2026-09-15	Bengaluru East Hub	Hyderabad Central DC
7	IN_TRANSIT	2026-09-22	Hyderabad Central DC	Pune West Depot
8	PENDING	2026-10-01	Pune West Depot	Hyderabad Central DC

SQL activity

Clear

Statements sent by inserts, deletes and workflows appear here.

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46

8 rows

Figure 11.3 – Transfers table with status labels and warehouse names instead of IDs

11.3 Insert – before and after

File	Edit	View	Help
New transfer			
Process transfers	4		
Dispatch stock			
Record damage			
LOOK UP			
Bin stock	19		
Bin search			
Reports			
MOVEMENT RECORDS			
Receipts	9		
Receipt lines	16		
Transfers	8		
Transfer lines	8		
Dispatches	6		
Dispatch lines	11		
Damage records	5		
LOCATIONS			
Warehouses	3		
Zones	7		
Bins	17		
PRODUCTS & SUPPLIERS			
Products	13		
Suppliers	7		

Suppliers

Companies that deliver stock.
Table supplier · key (supplier_id)

Refresh Export CSV Delete New

All columns Filter rows (Ctrl+F)

ID	Name	Contact
1	Deccan Electricals Pvt Ltd	+91 40 4012 3456
2	Sahyadri Pipes & Fittings	+91 20 2567 8812
3	Kaveri Paints Co.	+91 80 4190 2275
4	Vajra Tools & Hardware	+91 22 2854 6630
5	Nilgiri Cables & Wires	+91 422 245 7781
6	Godavari Sanitary Fittings	+91 891 276 4410
7	Malwa Switchgear Pvt Ltd	+91 731 249 3367

SQL activity Clear

Statements sent by inserts, deletes and workflows appear here.

7 rows

Figure 11.4 – Before: Suppliers table with 7 rows

```
mysql> SELECT * FROM supplier;
```

supplier_id	name	contact
1	Deccan Electricals Pvt Ltd	+91 40 4012 3456
2	Sahyadri Pipes & Fittings	+91 20 2567 8812
3	Kaveri Paints Co.	+91 80 4190 2275
4	Vajra Tools & Hardware	+91 22 2854 6630
5	Nilgiri Cables & Wires	+91 422 245 7781
6	Godavari Sanitary Fittings	+91 891 276 4410
7	Malwa Switchgear Pvt Ltd	+91 731 249 3367

New supplier

Inserts one row into table supplier

Name

Contact

Phone number, e.g. +91 40 4012 3456

Cancel Insert

Figure 11.5 – Insert form generated from the schema

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage

LOOK UP
Bin stock 19
Bin search
Reports

MOVEMENT RECORDS
Receipts 9
Receipt lines 16
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5

LOCATIONS
Warehouses 3
Zones 7
Bins 17

PRODUCTS & SUPPLIERS
Products 13
Suppliers 8

Suppliers

Companies that deliver stock.
Table supplier · key (supplier_id)

Refresh Export CSV Delete New

All columns Filter rows (Ctrl+F)

ID	Name	Contact
1	Deccan Electricals Pvt Ltd	+91 40 4012 3456
2	Sahyadri Pipes & Fittings	+91 20 2567 8812
3	Kaveri Paints Co.	+91 80 4190 2275
4	Vajra Tools & Hardware	+91 22 2854 6630
5	Nilgiri Cables & Wires	+91 422 245 7781
6	Godavari Sanitary Fittings	+91 891 276 4410
7	Malwa Switchgear Pvt Ltd	+91 731 249 3367
8	Shakti Hardware Pvt Ltd	+91 44 2345 6789

SQL activity Clear

```
05:05:02 START TRANSACTION
05:05:02 INSERT INTO `supplier` (`name`, `contact`) VALUES (%s, %s) ['Shakti Hardware Pvt Ltd', '+91 44 2345 6789'] 1 row affected
05:05:02 COMMIT
```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Inserted supplier "Shakti Hardware Pvt Ltd". 8 rows

Figure 11.6 – After: new supplier (ID 8) selected, count 7 → 8, SQL panel shows INSERT and COMMIT

```
mysql> SELECT * FROM supplier;
+-----+-----+-----+
| supplier_id | name | contact |
+-----+-----+-----+
| 1 | Deccan Electricals Pvt Ltd | +91 40 4012 3456 |
| 2 | Sahyadri Pipes & Fittings | +91 20 2567 8812 |
| 3 | Kaveri Paints Co. | +91 80 4190 2275 |
| 4 | Vajra Tools & Hardware | +91 22 2854 6630 |
| 5 | Nilgiri Cables & Wires | +91 422 245 7781 |
| 6 | Godavari Sanitary Fittings | +91 891 276 4410 |
| 7 | Malwa Switchgear Pvt Ltd | +91 731 249 3367 |
| 8 | Shakti Hardware Pvt Ltd | +91 44 2345 6789 |
+-----+-----+-----+
```

New receipt line

Inserts one row into table receipt_line

Receipt #3 2026-08-07 Kaveri Paints Co. → Pune W

Bin PUN-BS-02 Pune West Depot / Bulk Storage

Product PNT-WHT-020 Wall Paint 20L White

Quantity received 50

Whole units, greater than zero

Cancel Insert

Figure 11.7 – Inserting a receipt line of 50 × Wall Paint into bin PUN-BS-02 (foreign keys chosen from dropdowns)

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage

LOOK UP

Bin stock 20
Bin search
Reports

MOVEMENT RECORDS
Receipts 9
Receipt lines 17
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5

LOCATIONS
Warehouses 3
Zones 7
Bins 17

PRODUCTS & SUPPLIERS
Products 13
Suppliers 8

Bin stock

Current stock in every bin, calculated from all movements (read-only).

View `v_bin_stock` · key (bin_id, product_id)

All columns
Filter rows (Ctrl+F)

Refresh
Export CSV

On hand total **5,176**

Warehouse	Bin	SKU	Product	On hand	Received	Transfer in	Tran
Hyderabad Central DC	HYD-BS-01	ELE-LED-009	LED Bulb 9W (Pack of 4)	726	1,200	0	
Hyderabad Central DC	HYD-BS-02	ELE-SWT-006	Modular Switch 6A	1,600	2,500	0	
Hyderabad Central DC	HYD-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	50	120	0	
Hyderabad Central DC	HYD-FP-01	ELE-MCB-032	MCB 32A Single Pole	130	400	0	
Hyderabad Central DC	HYD-FP-02	TLS-BIT-005	Hammer Drill Bit Set 5pc	70	70	0	
Hyderabad Central DC	HYD-FP-02	TLS-DRL-018	Cordless Drill 18V	38	40	0	
Bengaluru East Hub	BLR-BS-01	PLB-PVC-025	PVC Pipe 1in x 3m	588	600	0	
Bengaluru East Hub	BLR-BS-02	ELE-LED-009	LED Bulb 9W (Pack of 4)	300	0	300	
Bengaluru East Hub	BLR-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	40	0	40	
Bengaluru East Hub	BLR-BS-03	PLB-ELB-025	CPVC Elbow 1in (Pack of 10)	180	300	0	
Bengaluru East Hub	BLR-FP-01	PLB-VAL-025	Ball Valve 1in Brass	35	90	0	

SQL activity
Clear

```

05:05:02 INSERT INTO `supplier` (`name`, `contact`) VALUES (%s, %s) ['Deccan Electricals Pvt Ltd', '+91 40 0000 0000']
05:05:02 ERROR 1062: Duplicate entry 'Deccan Electricals Pvt Ltd' for key 'supplier.uq_supplier_name'
05:05:02 ROLLBACK
05:05:03 START TRANSACTION
05:05:03 INSERT INTO `receipt_line` (`receipt_id`, `bin_id`, `product_id`, `quantity`) VALUES (%s, %s, %s, %s) [3, 14, 5, 50] 1 row affected
05:05:03 COMMIT

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46
Inserted receipt #3 line: 50 × PNT-WHT-020 into PUN-BS-02.
20 rows

Figure 11.8 – After: Bin stock now 20 rows, on-hand total 5,176

```
mysql> SELECT COUNT(*) AS bin_rows, SUM(on_hand) AS on_hand_total FROM v_bin_stock;
+-----+-----+
| bin_rows | on_hand_total |
+-----+-----+
|      20 |          5176 |
+-----+-----+
```

11.4 Delete – before and after

Delete receipt line?

You are about to delete

Receipt #3 line: 50 × PNT-WHT-020 into PUN-BS-02

Runs DELETE on table receipt_line in one transaction. This cannot be undone.

Cancel
Delete

Figure 11.9 – Delete confirmation names the exact record

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage

LOOK UP

Bin stock 19
Bin search
Reports

MOVEMENT RECORDS
Receipts 9
Receipt lines 16
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5

LOCATIONS
Warehouses 3
Zones 7
Bins 17

PRODUCTS & SUPPLIERS
Products 13
Suppliers 8

Bin stock

Current stock in every bin, calculated from all movements (read-only).

View `v_bin_stock` · key (bin_id, product_id)

All columns
Filter rows (Ctrl+F)

On hand total 5,126

Warehouse	Bin	SKU	Product	On hand	Received	Transfer in	Tran
Hyderabad Central DC	HYD-BS-01	ELE-LED-009	LED Bulb 9W (Pack of 4)	726	1,200	0	
Hyderabad Central DC	HYD-BS-02	ELE-SWT-006	Modular Switch 6A	1,600	2,500	0	
Hyderabad Central DC	HYD-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	50	120	0	
Hyderabad Central DC	HYD-FP-01	ELE-MCB-032	MCB 32A Single Pole	130	400	0	
Hyderabad Central DC	HYD-FP-02	TLS-BIT-005	Hammer Drill Bit Set 5pc	70	70	0	
Hyderabad Central DC	HYD-FP-02	TLS-DRL-018	Cordless Drill 18V	38	40	0	
Bengaluru East Hub	BLR-BS-01	PLB-PVC-025	PVC Pipe 1in x 3m	588	600	0	
Bengaluru East Hub	BLR-BS-02	ELE-LED-009	LED Bulb 9W (Pack of 4)	300	0	300	
Bengaluru East Hub	BLR-BS-03	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	40	0	40	
Bengaluru East Hub	BLR-BS-03	PLB-ELB-025	CPVC Elbow 1in (Pack of 10)	180	300	0	
Bengaluru East Hub	BLR-FP-01	PLB-VAL-025	Ball Valve 1in Brass	35	90	0	

SQL activity

Clear

```

05:05:03 INSERT INTO `transfer_line` (`transfer_id`, `source_bin_id`, `product_id`, `quantity`) VALUES (%s, %s, %s, %s) [1, 4, 7, 5] ERROR 1644: Lines on a CONFIRMED transfer must have a dest_bin_id
05:05:03 ROLLBACK
05:05:03 START TRANSACTION
05:05:03 DELETE FROM `receipt_line` WHERE `receipt_id` = %s AND `bin_id` = %s AND `product_id` = %s [3, 14, 5] 1 row affected
05:05:03 COMMIT

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Deleted receipt #3 line: 50 × PNT-WHT-020 into PUN-BS-02. 19 rows

Figure 11.10 – After deleting the receipt line: Bin stock back to 19 rows, 5,126

```
mysql> SELECT COUNT(*) AS bin_rows, SUM(on_hand) AS on_hand_total FROM v_bin_stock;
+-----+-----+
| bin_rows | on_hand_total |
+-----+-----+
|      19 |          5126 |
+-----+-----+
```

Delete supplier?

You are about to delete

Supplier "Shakti Hardware Pvt Ltd"

Runs DELETE on table supplier in one transaction. This cannot be undone.

Cancel
Delete

Figure 11.11 – Deleting the supplier added earlier

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage

LOOK UP
Bin stock 19
Bin search
Reports

MOVEMENT RECORDS
Receipts 9
Receipt lines 16
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5

LOCATIONS
Warehouses 3
Zones 7
Bins 17

PRODUCTS & SUPPLIERS
Products 13
Suppliers 7

Suppliers

Companies that deliver stock.

Table `supplier` · key (supplier_id)

All columns
Filter rows (Ctrl+F)

ID	Name	Contact
1	Deccan Electricals Pvt Ltd	+91 40 4012 3456
2	Sahyadri Pipes & Fittings	+91 20 2567 8812
3	Kaveri Paints Co.	+91 80 4190 2275
4	Vajra Tools & Hardware	+91 22 2854 6630
5	Nilgiri Cables & Wires	+91 422 245 7781
6	Godavari Sanitary Fittings	+91 891 276 4410
7	Malwa Switchgear Pvt Ltd	+91 731 249 3367

RefreshExport CSVDeleteNew

SQL activityClear

```

05:05:03 START TRANSACTION
05:05:03 DELETE FROM `receipt_line` WHERE `receipt_id` = %s AND `bin_id` = %s AND `product_id` = %s [3, 14, 5] 1
row affected
05:05:03 COMMIT
05:05:03 START TRANSACTION
05:05:03 DELETE FROM `supplier` WHERE `supplier_id` = %s [8] 1 row affected
05:05:03 COMMIT

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Deleted supplier "Shakti Hardware Pvt Ltd". 7 rows

Figure 11.12 – After: Suppliers back to 7 rows

```
mysql> SELECT * FROM supplier;
```

supplier_id	name	contact
1	Deccan Electricals Pvt Ltd	+91 40 4012 3456
2	Sahyadri Pipes & Fittings	+91 20 2567 8812
3	Kaveri Paints Co.	+91 80 4190 2275
4	Vajra Tools & Hardware	+91 22 2854 6630
5	Nilgiri Cables & Wires	+91 422 245 7781
6	Godavari Sanitary Fittings	+91 891 276 4410
7	Malwa Switchgear Pvt Ltd	+91 731 249 3367

Delete warehouse?

You are about to delete

Warehouse "Hyderabad Central DC"

This also deletes 3 zones and 7 bins (ON DELETE CASCADE).

Runs DELETE on table warehouse in one transaction. This cannot be undone.

CancelDelete

Figure 11.13 – Cascade preview when deleting a warehouse (3 zones and 7 bins)

11.5 Other pages

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage
LOOK UP
Bin stock 19
Bin search
Reports
MOVEMENT RECORDS
Receipts 9
Receipt lines 16
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5
LOCATIONS
Warehouses 3
Zones 7
Bins 17
PRODUCTS & SUPPLIERS
Products 13
Suppliers 7

Reports

Summaries of stock, transfers, bins, ageing, damage and reorder needs. Nothing is changed here.
Choose a tab. Ctrl+F filters the rows; Ctrl+E saves the rows shown as a CSV file.

Warehouse stock Pending and in transit Bin utilization Stock ageing Damaged stock Reorder needs

Units on hand per warehouse and product (from v_bin_stock); reserved = held by PENDING transfers.

Filter rows (Ctrl+F) On hand 5,126 · Reserved 165 · Available 4,961

Warehouse	SKU	Product	Bins	On hand	Reserved	Available
Bengaluru East Hub	ELE-LED-009	LED Bulb 9W (Pack of 4)	1	300	0	300
Bengaluru East Hub	ELE-WIR-015	Copper Wire 1.5 sq mm (90m)	1	40	0	40
Bengaluru East Hub	PLB-ELB-025	CPVC Elbow 1in (Pack of 10)	1	180	0	180
Bengaluru East Hub	PLB-PVC-025	PVC Pipe 1in x 3m	1	588	150	438
Bengaluru East Hub	PLB-VAL-025	Ball Valve 1in Brass	1	35	0	35
Bengaluru East Hub	PNT-BLK-001	Enamel Paint 1L Black	1	114	0	114
Bengaluru East Hub	PNT-WHT-020	Wall Paint 20L White	1	55	0	55
Hyderabad Central DC	ELE-LED-009	LED Bulb 9W (Pack of 4)	1	726	0	726
Hyderabad Central DC	ELE-MCB-032	MCB 32A Single Pole	1	130	0	130

SQL activity Clear

```

05:05:03 DELETE FROM `supplier` WHERE `supplier_id` = %s [8] 1 row affected
05:05:03 COMMIT
05:05:04 START TRANSACTION
05:05:04 DELETE FROM `supplier` WHERE `supplier_id` = %s [1] ERROR 1451: Cannot delete or update a parent row: a
foreign key constraint fails (`stock_transfer_db`.`receipt`, CONSTRAINT `fk_receipt_supplier` FOREIGN KEY
(`supplier_id`) REFERENCES `supplier` (`supplier_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
05:05:04 ROLLBACK

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Delete refused for supplier "Deccan Electricals Pvt Ltd". It is still referenced by receipts. Delete tho 19 rows

Figure 11.14 – Reports: warehouse stock (on hand, reserved, available)

File Edit View Help

New transfer
Process transfers 4
Dispatch stock
Record damage
LOOK UP
Bin stock 19
Bin search
Reports
MOVEMENT RECORDS
Receipts 9
Receipt lines 16
Transfers 8
Transfer lines 8
Dispatches 6
Dispatch lines 11
Damage records 5
LOCATIONS
Warehouses 3
Zones 7
Bins 17
PRODUCTS & SUPPLIERS
Products 13
Suppliers 7

Bin search

Find bins and what they hold by bin code, zone, SKU or product name.
Reserved = promised to pending transfers. Available = on hand – reserved. · Reads bin joined to v_bin_stock

All warehouses pvc ☐ Include empty bins 2 bins · On hand total 768

Warehouse	Zone	Bin	SKU	Product	On hand	Reserved	Available
Bengaluru East Hub	Bulk Storage	BLR-BS-01	PLB-PVC-025	PVC Pipe 1in x 3m	588	150	438
Bengaluru East Hub	Bulk Storage	BLR-BS-03	PLB-ELB-025	CPVC Elbow 1in (Pack of 10)	180	0	180

SQL activity Clear

```

05:05:03 DELETE FROM `supplier` WHERE `supplier_id` = %s [8] 1 row affected
05:05:03 COMMIT
05:05:04 START TRANSACTION
05:05:04 DELETE FROM `supplier` WHERE `supplier_id` = %s [1] ERROR 1451: Cannot delete or update a parent row: a
foreign key constraint fails (`stock_transfer_db`.`receipt`, CONSTRAINT `fk_receipt_supplier` FOREIGN KEY (`supplier_id`)
REFERENCES `supplier` (`supplier_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
05:05:04 ROLLBACK

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Delete refused for supplier "Deccan Electricals Pvt Ltd". It is still referenced by receipts. Delete those rece 2 rows

Figure 11.15 – Bin search for "pvc" with reserved and available quantities

File Edit View Help

New transfer

Process transfers 4

Dispatch stock

Record damage

LOOK UP

Bin stock 19

Bin search

Reports

MOVEMENT RECORDS

Receipts 9

Receipt lines 16

Transfers 8

Transfer lines 8

Dispatches 6

Dispatch lines 11

Damage records 5

LOCATIONS

Warehouses 3

Zones 7

Bins 17

PRODUCTS & SUPPLIERS

Products 13

Suppliers 7

Process transfers

Ship, put away and confirm transfers that are still open.

1 Select a transfer 2 Mark in transit when it leaves 3 Choose a destination bin for each line 4 Confirm receipt when it arrives · Updates transfer.status, transfer_line.dest_bin_id

OPEN TRANSFERS (PENDING AND IN_TRANSIT)

Transfer	Status	Date	From	To	Lines	Units	Lines without de
2	IN_TRANSIT	2026-08-14	Hyderabad Central DC	Pune West Depot	1	500	
3	PENDING	2026-08-18	Bengaluru East Hub	Pune West Depot	1	150	
7	IN_TRANSIT	2026-09-22	Hyderabad Central DC	Pune West Depot	1	150	
8	PENDING	2026-10-01	Pune West Depot	Hyderabad Central DC	1	15	

LINES AND PUT-AWAY

Transfer #2 · IN_TRANSIT · Hyderabad Central DC → Pune West Depot

Source bin	SKU	Product	Quantity	Destination bin

Cancel transfer

Save put-away bins

Mark in transit

Confirm receipt

SQL activity

Clear

```

05:05:03 DELETE FROM `supplier` WHERE `supplier_id` = %s [8] 1 row affected
05:05:03 COMMIT
05:05:04 START TRANSACTION
05:05:04 DELETE FROM `supplier` WHERE `supplier_id` = %s [1] ERROR 1451: Cannot delete or update a parent row: a
foreign key constraint fails (`stock_transfer_db`.`receipt`, CONSTRAINT `fk_receipt_supplier` FOREIGN KEY (`supplier_id`)
REFERENCES `supplier` (`supplier_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
05:05:04 ROLLBACK

```

root@localhost:3306 / stock_transfer_db · MySQL 8.0.46 Delete refused for supplier "Deccan Electricals Pvt Ltd". It is still referenced by receipts. Delete those receipts fir

Figure 11.16 – Process transfers: mark in transit, choose destination bins, confirm or cancel

12. Implementation Details

12.1 Technology stack

Layer	Technology
User interface	Python 3 · PySide6 (Qt 6), Fusion style, one stylesheet
Business rules and workflows	operations.py
Data access	db.py · mysql-connector-python (all SQL, transactions, error translation)
Database	MySQL 8.0 · stock_transfer_db (constraints, triggers, view)

```

main.py — LoginDialog — credentials.py (Windows Credential Manager)
|
MainWindow — Sidebar · SQL activity panel · status bar
  |— TablePage ×13 — RecordDialog (insert) · ConfirmDeleteDialog
  |— workflows/ ×6 — receive · new transfer · process transfers · dispatch · damage · bin
search
  |— ReportsPage — 6 reports · CSV export
  |
  operations.py (business rules, workflows)
  |
  db.py (every SQL statement, connection, transactions, error translation)
  |
  mysql-connector-python — MySQL 8.0 · stock_transfer_db

```

No SQL is written in the UI layer. Every value is passed as a %s parameter, which prevents SQL injection; table and column names come only from the schema definitions.

12.2 Database connectivity

```

def _open(info: ConnectionInfo, password: str) -> Any:
    try:
        return mysql.connector.connect(
            host=info.host, port=info.port, user=info.user, password=password,
            database=info.database, autocommit=True, charset="utf8mb4",
            collation="utf8mb4_0900_ai_ci", connection_timeout=CONNECT_TIMEOUT_SECONDS,
        )
    except mysql.connector.Error as err:
        raise DbError(_connect_message(err, info), code=err.errno) from err

```

12.3 Transactions

```

@contextmanager
def transaction(self) -> Iterator[Transaction]:
    """Run the block as one transaction: commit on success, roll back on any error."""
    self._ensure_connected()
    self._conn.start_transaction()
    self._emit(StatementRecord("START TRANSACTION"))
    try:
        yield Transaction(self._conn, self._emit)
        self._conn.commit()
        self._emit(StatementRecord("COMMIT"))
    except mysql.connector.Error as err:
        self._rollback()
        raise translate_error(err) from err

```

12.4 Parameterized insert

```
def insert_values(tx: Transaction, spec: TableSpec, values: dict[str, Any]) -> tuple[Any, ...]:
    allowed = {f.column for f in spec.fields}
    columns = [column for column in values if column in allowed]
    if len(columns) != len(values):
        raise ValueError(f"Unknown column for {spec.name}: {set(values) - allowed}")
    column_list = ", ".join(f"`{column}`" for column in columns)
    placeholders = ", ".join(["%s"] * len(columns))
    sql = f"INSERT INTO `{spec.name}` ({column_list}) VALUES ({placeholders})"
    _, last_id = tx.execute(sql, [values[column] for column in columns])
```

12.5 A complete workflow in one transaction

```
def receive(database, supplier_id, warehouse_id, receipt_date, lines) -> int:
    """Post a receipt and put its lines away into bins of the receiving warehouse."""
    _require_lines(lines)
    _check_bins(database, lines, warehouse_id)
    with database.transaction() as tx:
        (receipt_id,) = db.insert_values(tx, schema.RECEIPT,
            {"supplier_id": supplier_id, "warehouse_id": warehouse_id, "receipt_date":
receipt_date})
        for line in lines:
            db.insert_values(tx, schema.RECEIPT_LINE,
                {"receipt_id": receipt_id, "bin_id": line.bin_id,
                 "product_id": line.product_id, "quantity": line.quantity})
    return int(receipt_id)
```

12.6 MySQL errors shown in plain language

MySQL error	Meaning	What the user sees
1062	Duplicate key	"A supplier with this name already exists." under the field
1451	Row is referenced	"It is still referenced by receipts. Delete those receipts first."
1452	Parent missing	"The selected warehouse no longer exists."
3819	CHECK violated	The rule in words, e.g. "Source and destination warehouse must be different."
1644	Trigger SIGNAL	The trigger message unchanged

12.7 Rules added by the application

- A bin on a receipt, transfer or dispatch must belong to that document's warehouse.
- A transfer or dispatch cannot take more than the available stock (on hand – reserved by PENDING transfers).
- Transfers move forward only: PENDING → IN_TRANSIT → CONFIRMED, or CANCELLED; CONFIRMED and CANCELLED are final.
- A delete may never leave a bin with negative stock.

13. Testing

13.1 Test cases and results

#	Test	Input	Expected	Result
T1	View bin stock	Open Bin stock	19 rows, total 5,126	Pass
T2	Insert supplier	Shakti Hardware Pvt Ltd	Row added, 7 → 8 in app and MySQL	Pass
T3	Insert receipt line	Receipt 3, PUN-BS-02, 50 units	Bin stock 20 rows, 5,176	Pass
T4	Delete receipt line	Row from T3	Bin stock back to 19 rows, 5,126	Pass
T5	Delete supplier	Supplier from T2	Suppliers back to 7	Pass
T6	Duplicate supplier name	Deccan Electricals Pvt Ltd	Rejected, MySQL 1062	Pass
T7	Duplicate bin code	HYD-BS-01 again	Rejected, MySQL 1062	Pass
T8	Zero quantity	Receipt line quantity 0	Rejected, MySQL 3819	Pass
T9	Same source and destination warehouse	Transfer 1 → 1	Rejected, MySQL 3819 / app rule	Pass
T10	Confirm transfer before put-away	Transfer 2 → CONFIRMED	Rejected by trigger, MySQL 1644	Pass
T11	Line on a CONFIRMED transfer without destination bin	Transfer 1, no dest bin	Rejected by trigger, MySQL 1644	Pass
T12	Delete supplier still referenced	Deccan Electricals	Refused, MySQL 1451, ROLLBACK	Pass
T13	Delete warehouse	Hyderabad Central DC	Dialog previews 3 zones and 7 bins cascading	Pass
T14	Transfer more than available	800 from 726 available	Refused by application	Pass

13.2 Evidence

New supplier

Inserts one row into table supplier

Name

Deccan Electricals Pvt Ltd

A supplier with this name already exists.

Contact

+91 40 0000 0000

Phone number, e.g. +91 40 4012 3456

Cancel

Insert

Figure 13.1 – T6: duplicate name shown under the field

New transfer line

Inserts one row into table transfer_line

Transfer	#1 CONFIRMED Hyderabad Central DC
Source bin	HYD-BS-02 Hyderabad Central DC / Bul
Product	ELE-SWT-006 Modular Switch 6A
Quantity	5
Whole units, greater than zero	
Destination bin (optional)	— None —
Leave empty until put-away; required once the transfer is confirmed	

Lines on a CONFIRMED transfer must have a dest_bin_id
MySQL 1644: rejected by a trigger (SQLSTATE 45000)

Cancel Insert

Figure 13.2 – T11: trigger trg_transfer_line_dest_bi rejects the line

Could not delete

Could not delete supplier "Deccan Electricals Pvt Ltd". It is still referenced by receipts. Delete those receipts first.

MySQL 1451: Cannot delete or update a parent row: a foreign key constraint fails (`stock\_transfer\_db`.`receipt`, CONSTRAINT `fk\_receipt\_supplier` FOREIGN KEY (`supplier\_id`) REFERENCES `supplier` (`supplier\_id`) ON DELETE RESTRICT ON UPDATE CASCADE)

Close

Figure 13.3 – T12: delete refused because receipts still reference the supplier

Constraint tests run directly at the MySQL prompt (T7–T10):

```
INSERT INTO Bin (bin_code, zone_id) VALUES ('HYD-BS-01',2);
```

```
ERROR 1062 (23000) at line 1: Duplicate entry 'HYD-BS-01' for key 'bin.uq_bin_code'
```

```
INSERT INTO Receipt_Line VALUES (1,3,1,0);
```

```
ERROR 3819 (HY000) at line 1: Check constraint 'chk_receipt_line_qty' is violated.
```

```
INSERT INTO Transfer (source_warehouse_id,dest_warehouse_id) VALUES (1,1);
```

```
ERROR 3819 (HY000) at line 1: Check constraint 'chk_transfer_diff_warehouse' is violated.
```

```
UPDATE Transfer SET status='CONFIRMED' WHERE transfer_id=2;
```

```
ERROR 1644 (45000) at line 1: Cannot confirm transfer: every line needs a dest_bin_id (put-away) first
```

14. Conclusion and Future Enhancements

The project delivered, across three reviews, a conceptual ER design, a normalized MySQL database with keys, CHECK constraints, triggers and a derived stock view, and a working desktop application connected live to it. Insertion, deletion and viewing of records were demonstrated with every change visible both in the application and at the MySQL prompt, and every business rule was tested. Because stock is derived from movements instead of stored, the balances cannot drift, and stock in transit is never counted twice.

Future enhancements

- Move the application-side stock checks (quantity within available stock, bin belongs to the warehouse) into database triggers so that every client is bound by them.
- User accounts and roles matching the four user types, with permissions enforced in MySQL.
- Bin capacity, so utilisation can be measured in units and not only occupied bins.
- Barcode scanning for bins and products.
- More reports and dashboards; a web or mobile front end.

15. References

1. Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
2. Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
3. Oracle. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
4. Oracle. *MySQL Connector/Python Developer Guide*. <https://dev.mysql.com/doc/connector-python/en/>
5. The Qt Company. *Qt for Python (PySide6) Documentation*. <https://doc.qt.io/qtforpython-6/>
6. DBMS Course Project Instructions, AIML Whales and Panthers, Woxsen University, 2026.

16. Appendix: GitHub Repository Link

<https://github.com/ShreenandhK/DBMS-Course-Project.git>

Folder / file	Contents
Presentation-I	Problem description presentation
Presentation-II	Review 2 presentation, ER diagram, given query and its output
Presentation-III	Review 3 presentation, UI screenshots
Project-Report	This report (PDF)
UI/	Application source code (Python, PySide6)
stock_transfer_db.sql	DDL, constraints, triggers, view and sample data
verify_stock_transfer_db.sql	Verification queries
README.md	Name, roll number, project title and description